

Support-Aware Neuron Research Plan

Complete PDF plan for a new neural network primitive in which response and support are intrinsic, structurally separate, and experimentally falsifiable.

This research plan was originated and developed by Alberto Popescu on April 5, 2026. The core primitive - support-aware neurons with intrinsic local and joint support, propagated certainty, and support-gated plasticity - was conceived through the reasoning process documented in the originating conversation.

Purpose. This document captures the full research plan for a support-aware neural network primitive. It defines the primitive, states the design contract, specifies falsifiability conditions, adds the required A7 scalar-gate control, and lays out the smallest experimental build needed to test the core claim in a compositional novelty setting.

Core claim	A neuron should carry both a response and an intrinsic record of support from training experience, and that support should shape both forward trust and backward plasticity.
First decisive experiment	Regime 2: unseen combinations of familiar parts. This is the fastest way to test whether support tracks encountered existence rather than feature popularity or generic difficulty.
Scientific standard	The idea is only retained if it survives explicit falsifiability conditions, controlled ablations, and negative predictions in IID-saturated settings.

1. Foundation: the primitive being proposed

The proposal does not add a certainty feature to an otherwise standard transformer. It changes what a computational unit is allowed to be. A standard neuron maps inputs to a response but carries no intrinsic record of how often similar states have been encountered during training. The proposed primitive adds that missing ontological piece: support.

neuron = (response, support)

Response answers what pattern is being detected. Support answers how much training experience exists for states like this. These quantities are separate by definition and must remain separate throughout forward computation and backward learning.

support = (local support, joint support)

Local support belongs to each neuron. Joint support belongs to each layer and captures whether the current combination of activations has been encountered before. Certainty is inherited across layers, updated by current support, and then used to influence both trust in the forward pass and plasticity in the backward pass.

2. One-line anchor definition

A support-aware neuron is a computational unit that learns a response through weights, accumulates support through experience, passes both forward, and uses support to control how strongly errors rewrite it.

Every design choice in the architecture should be checked against that sentence. If a choice cannot be justified by it, it does not belong in the first implementation.

3. Design contract: non-negotiable constraints

The architecture should only be considered faithful if it satisfies the following ten constraints. These are not implementation preferences; they are the design contract.

Point	Constraint
1	Certainty is not a module. It is a property of every neuron.
2	Certainty comes from counting existence, not from learning. No certainty loss, no supervision, no meta-objective.
3	Response and certainty stay structurally separate. They may interact through an operator, but they must never collapse into

	one vector.
4	Certainty includes combination-level existence, not just neuron-level existence.
5	Certainty propagates across layers. It is inherited and updated, not recomputed independently.
6	Certainty modulates backprop without being trained by it.
7	Warmup and activation phases are criterion-based, not time-based.
8	No bilevel optimization, no meta-learning, no learned plasticity codes.
9	With neutral certainty, behavior should match a baseline transformer.
10	Removing certainty should not leave a complete faithful model behind; it should leave something architecturally incomplete.

4. Working architecture

4.1 Local support at the neuron level

Each neuron i stores a learned weight vector w_i and a persistent local support state s_i . During training, local support updates directly from experience rather than from gradients.

$$a_i(x) = \phi(w_i^T x + b_i)$$

$$s_i^{(t+1)} = \lambda s_i^{(t)} + (1 - \lambda) \rho(a_i(x_t))$$

The function ρ is a soft measure of meaningful activation. Decay λ prevents support from saturating permanently and lets support track recent experience rather than lifetime raw counts.

4.2 Joint support at the layer level

Novelty often lives in combinations of familiar parts. To capture that, each layer maintains a compressed memory of joint activation patterns. This can be implemented with a fixed random sketch of the layer activation vector plus running mean and variance in sketch space.

$$z_l = R_l a_l$$

$$\begin{aligned}\mu_l^{(t+1)} &= \lambda_\mu \mu_l^{(t)} + (1 - \lambda_\mu) z_l \\ v_l^{(t+1)} &= \lambda_v v_l^{(t)} + (1 - \lambda_v) (z_l - \mu_l)^2 \\ d_l &= \sum_j (z_{\{l,j\}} - \mu_{\{l,j\}})^2 / (v_{\{l,j\}} + \epsilon)\end{aligned}$$

Low distance means the current joint activation pattern resembles previously encountered combinations. High distance means the layer has entered a novel configuration even if many individual neurons are familiar.

4.3 Support propagation across layers

Support is inherited, not recomputed from scratch. A token that enters unfamiliar territory in an earlier layer should carry that unfamiliarity forward.

$$c_l(x) = \Psi(c_{\{l-1\}}(x), c_l^{\text{joint}}(x), \{c_i^{\text{local}}(x)\}_{i \in l})$$

In practice, the easiest stable implementation is to propagate certainty in logit space: inherit the previous layer logit, add current evidence, then read certainty through a bounded nonlinearity.

4.4 Relational operator between response and support

Response and support must not collapse into a scalar product by default. The faithful architecture uses a relational operator Ω that keeps response and support distinct and lets meaning emerge from their interaction.

$$\tilde{a}_i = \Omega(a_i, c_i, c_{\text{context}})$$

The scalar gate version is still important as an ablation, but it is not the strongest architectural claim. The relational operator is the part that tests whether support and response need to interact more richly than simple trust scaling.

4.5 Backward plasticity

Error determines the direction of change. Support determines the authority of the example to rewrite the weights.

$$\nabla w_i \leftarrow \nabla w_i \cdot h(e_i, c_i)$$

The update rule should be asymmetric: high-support wrong cases receive the strongest corrective updates; low-support wrong cases receive smaller cautious updates; low-support right cases receive consolidating updates; high-support right cases move minimally.

5. Architectural update: support and predictability as pre-answer priors

The original support-aware design computed support alongside the answer and then used that support to gate forward trust and backward plasticity. The updated architecture reverses that order. Support and predictability are computed before the answer and together form a prior over the prediction space.

This matters because support is not just an annotation on a finished answer. It becomes part of the model's pre-decision state. The model first asks how familiar this input region is and how stable the input-output mapping has historically been in this region, and only then generates an answer inside that prior.

$p(x)$ = support prior

$m(x)$ = predictability prior

The support prior $p(x)$ captures encountered existence: how much local and joint training support exists around the current input configuration. The predictability prior $m(x)$ captures how stable and directional the mapping has historically been in that region. Predictability is broader than monotonicity: monotonicity is one special case of a region whose input-output behavior is orderly and consistent.

$\pi(x) = \Omega_{\text{prior}}(p(x), m(x))$

$y(x) = F(x \mid \pi(x))$

Prediction is therefore conditioned on a prior estimate of the model’s own epistemic state. In high-support, high-predictability regions the model can use sharper routing, narrower answer spaces, and stronger commitment. In low-support or low-predictability regions it should widen the candidate set, allocate more computation, and avoid overcommitment.

$\delta(x) = D(\text{posterior behavior}, \pi(x))$

After prediction, the model compares the realized answer behavior against the prior that support and predictability defined. A narrow, confident answer in a weak-support region is a discrepancy. A hesitant answer in a high-support, high-predictability region is also a discrepancy. This gives the architecture a prior-posterior consistency signal that the older “predict then measure” design could not express.

Architecturally, the update can be stated in one sentence: the model does not merely attach uncertainty to an answer after it is formed; it conditions the act of prediction on an explicit internal estimate of its own support and predictability before the answer exists.

6. Falsifiability conditions

The theory is scientific only if it can fail cleanly. The following conditions define what would disconfirm the idea rather than merely weaken it.

Question	Prediction	Failure signal
When should the model be worse?	In fully IID, saturated settings it should match or slightly underperform a standard transformer because support adds structure where little novelty information exists.	If it wins everywhere, it is probably acting as a generic regularizer rather than as a support primitive.
Does support track existence rather than popularity?	Unseen combinations of familiar parts should receive lower support than seen combinations even when marginal feature frequency is	If final support is explained by feature popularity, token frequency, activation magnitude, or generic loss, the

	matched.	core claim fails.
Does support-gated learning use support correctly?	The four support/error buckets should show different update authority in the predicted order.	If gating just suppresses hard examples or behaves like generic gradient clipping, the learning claim fails.
Does joint support add value beyond local support?	Removing joint support should selectively hurt novel-combination cases more than rare-feature cases.	If local and joint support ablations look the same, the layer-level primitive is unnecessary.

7. Experimental program and ablations

Every meaningful result must be compared against a matched baseline and a targeted ablation set. The ablations are what make the theory interpretable.

ID	Ablation	Purpose
A1	No local support	Tests whether neuron-level support is needed for rare or weakly established local features.
A2	No joint support	Tests whether layer-level support adds anything beyond local support.
A3	No propagation	Tests whether inherited uncertainty across layers matters.
A4	No backward modulation	Tests whether support needs to shape learning rather than only forward trust.
A5	Neutral support control	Tests whether support values themselves carry information or the gains come from architectural side effects.
A6	Difficulty-weighting control	Tests whether the method is

		just a fancy version of hard-example weighting.
A7	Scalar gate control	Replaces Ω with $\tilde{a}_i = a_i \cdot c_i$. This tests whether the relational operator does real work beyond simple multiplicative trust scaling.
Baseline	Standard transformer	Reference point with matched parameter count, optimizer, data order, and training budget.

8. Regime-based evaluation plan

- Regime 1: IID saturated. Expect match or slight cost relative to baseline. This is the honest negative-prediction regime.
- Regime 2: compositional novelty. Familiar parts appear in unseen combinations at test time. This is the decisive first experiment.
- Regime 3: distribution shift. Tests whether support responds to out-of-support inputs without destructive over-updating.
- Regime 4: continual learning or drift. Tests whether support-sensitive plasticity improves the stability–plasticity tradeoff.

The idea is only compelling if its gains are regime-specific. A universal win across all settings would suggest a generic regularization effect rather than a genuine support primitive.

9. Minimal implementation plan for Regime 2

Do not build the full architecture first. Build the smallest system that can answer the core question: does final support fall on unseen combinations of familiar parts after controlling for feature frequency and difficulty?

Component	Minimal version required
Support-aware neuron	Persistent local support state updated by EMA from soft activation mass; no gradient path into support.
Joint support sketch	Fixed random projection of layer activations with EMA mean and variance; compute joint

	novelty from normalized sketch distance.
Propagation	Carry certainty forward across layers in explicit form rather than recomputing per layer.
Forward interaction	Implement both the full relational operator and the A7 scalar gate control.
Backward modulation	Use a fixed analytical support-gated update rule, not a learned gradient transform.
Readiness criteria	Turn on support-sensitive behavior only when support statistics stabilize by predefined measurable criteria.

10. Dataset and logging requirements for the first experiment

The first experiment should use a synthetic or controlled compositional dataset with two properties: the parts are individually common in training, and some combinations are deliberately withheld until test time.

- Recommended dataset types: colored-shape compositions, symbolic grammars, or templated sequence tasks with withheld structural pairings.
- For every test example, log correctness, model confidence, local support, joint support, propagated support, final support, seen/unseen combination label, and marginal part frequencies.
- Fit a post-hoc analysis predicting final support from marginal frequency, confidence, loss, local support, joint support, and the seen/unseen combination label.

Primary pass condition: after controlling for marginal frequency and difficulty, the seen/unseen combination label remains a strong predictor of final support. Primary fail condition: final support is mostly explained by feature popularity or generic difficulty.

11. Four-bucket learning analysis

Every implementation should report support-bucket behavior during training. This is the direct empirical test of the backward-plasticity claim.

Bucket	Interpretation	Expected update authority
High support + wrong	The model had support and still erred.	Strongest corrective update

Low support + wrong	The model was guessing in unfamiliar territory.	Cautious update
Low support + right	The model found something useful but not yet stabilized.	Moderate consolidating update
High support + right	The model behaved correctly in a familiar region.	Minimal update

12. Criterion-based warmup and activation

Support-sensitive behavior should never turn on simply because a certain number of optimization steps have passed. The model should activate it only when support statistics are stable enough to mean something.

- Local stabilization criterion: the distribution of per-neuron support changes by less than a predefined variance threshold over a fixed validation window.
- Joint stabilization criterion: layer-level joint support separates train-like samples from mild perturbations above a predefined AUC threshold.
- Neutral-certainty requirement: before these criteria are met, certainty remains behaviorally neutral so the model matches baseline transformer dynamics.

If support must be manually turned on by step counts tuned for each run, the architecture has not yet been implemented faithfully.

13. Risks, hidden assumptions, and honest failure modes

- The local support signal may collapse into feature popularity rather than epistemic support unless joint support really carries unique information.
- The relational operator may not outperform the scalar gate. If A7 matches the full model, the stronger architectural claim is unsupported.
- Joint support may be too weak or too noisy to detect novelty in realistic high-dimensional layers.
- Backward support gating may act like generic hard-example suppression if the four-bucket ordering does not materialize empirically.
- The architecture may add cost without benefit in regimes where novelty information is not needed. This is an expected negative result, not a bug in the scientific design.

14. Recommended order of work

Order	Action
-------	--------

1	Freeze this plan and preregistration, including A7 and the pass/fail conditions.
2	Implement only the minimal Regime 2 system: local support, joint support sketch, propagation, fixed backward rule, full Ω , and A7 control.
3	Run the compositional novelty experiment first. Do not scale up before this passes.
4	Only after Regime 2 succeeds, extend to broader evaluations: IID saturated, distribution shift, and continual learning.

15. Final scientific test

One-sentence falsifier. The support-aware neuron is falsified if its certainty can be explained by feature frequency, activation popularity, or example difficulty alone, rather than by encountered existence of local states and joint configurations, or if support-gated learning fails to selectively improve behavior in novelty-sensitive regimes.

This plan is intentionally strict. The purpose is not to maximize the chance of a positive result; it is to make the first experimental result interpretable. A clean negative result is more valuable than an ambiguous positive result produced by an overbuilt implementation.